

DataCORE

Filtering, Subsetting & Group Summaries in Pandas

Afternoon: DataCORE

- 1. Filtering & Subsetting — Asking Your Data a Question**
- 2. Handling Missing Values & Group-Level Summaries**
- 3. Scientific Storytelling — The One-Number Story**

This morning you named a target population and a DV. This afternoon we make the data answer for it.

1:40 – 2:45 · Filtering & Subsetting

Why filter? From the *whole* dataset to *your* question

- A dataset holds **everyone**. Your research question is about a **specific group**.
- This morning: maybe your population is **9th graders**, or **students who sleep less than 7 hours**.
- **Filtering** keeps only the rows that match — so every later number describes the group you actually care about.

Big idea: a filter is a **yes/no test** applied to every row. Rows that pass stay; rows that fail drop out.

 We continue with `study_habits.csv` — swap in **your** team file.

Reload the dataset

Python R

```
import pandas as pd
df = pd.read_csv("study_habits.csv") # rows = students, columns = variables
print(df.head())
```

	student_id	grade_level	study_method	hours_of_sleep	test_score
0	1	9th	solo	7.5	82.0
1	2	9th	group	6.0	74.0
2	3	9th	tutor	8.0	88.0
3	4	10th	solo	5.5	68.0
4	5	10th	group	7.0	79.0

 Columns: grade_level, study_method, hours_of_sleep, test_score.

The mask: a yes/no test for every row

Write the **condition** alone first — it returns `True/False` for each row.

Python R

```
mask = df["hours_of_sleep"] < 7    # a Boolean Series: one True/False per row
print(mask.head())
```

```
0    False
1     True
2    False
3     True
4    False
Name: hours_of_sleep, dtype: bool
```

 The mask is the **question**; the filter is the **answer**.

`df[df['col'] < value]` – filter to a subpopulation

Python R

```
under_7 = df[df["hours_of_sleep"] < 7]   # keep only rows that pass the test
print(under_7)
```

	student_id	grade_level	study_method	hours_of_sleep	test_score
1	2	9th	group	6.0	74.0
3	4	10th	solo	5.5	68.0
7	8	11th	group	6.5	77.0
10	11	11th	group	5.0	65.0
11	12	12th	solo	6.0	72.0
14	15	9th	solo	6.5	NaN
17	18	12th	solo	5.5	69.0
18	19	9th	group	6.0	75.0

= Use `==` for categories: `df[df["grade_level"] == "9th"]`.

Combine conditions with & (and) / | (or)

Python

R

```
# 9th graders who slept less than 7 hours -- wrap each condition in ()
subset = df[(df["grade_level"] == "9th") & (df["hours_of_sleep"] < 7)]
print(subset)
```

	student_id	grade_level	study_method	hours_of_sleep	test_score
1	2	9th	group	6.0	74.0
14	15	9th	solo	6.5	NaN
18	19	9th	group	6.0	75.0

Watch out: in Python use &/| (not and/or) and **parentheses** around each condition.

df.dropna() – handle missing values *intentionally*

Our data has blanks in `hours_of_sleep` and `test_score`. Drop rows missing a **key column** – not the whole table.

Python R

```
print("Before:", len(df))
```

Before: 20

```
clean = df.dropna(subset=["test_score"]) # drop rows with no DV value  
print("After: ", len(clean))
```

After: 19

 `subset=` matters: dropping *any* missing value can throw away rows you needed.

Compute the mean of your DV *for the subpopulation*

Python R

```
ninth = df[df["grade_level"] == "9th"]      # filter to the population
print(ninth["test_score"].mean())          # mean of the DV
```

79.75

 `na.rm` / pandas skips missing values automatically — know what you skipped.



df.groupby('category').mean() – all groups at once

Python

R

```
df.groupby("study_method")["test_score"].mean() # one mean per category
```

```
study_method
group      76.428571
solo       73.666667
tutor      91.833333
Name: test_score, dtype: float64
```



groupby does the filtering for *every* group in one line – instant comparison.

Team Challenge

Team Challenge – ~25 min

As a team, write code that:

1. Filters **your** dataset to your **target population**.
2. Drops rows missing your **DV** (*dropna*).
3. Computes the **mean of your DV**.

Instructor checks your output.

 Record your number **and its plain-English meaning** on your Mission Sheet.

Then discuss as a team:

What does this number actually mean **in real terms**?

Not "the mean is 78" – but "the average test score among 9th graders who slept under 7 hours was 78 out of 100."

AI prompt practice – then break it

1. Ask your AI tool:

"Write Python code to filter my DataFrame to rows where [column] equals [value]."

2. **Run** the code on your real data.

3. **Find the edge case it misses.**

Edge cases AI often forgets:

- Text vs. number: "9th" VS 9
- Capitalization: "Group" VS "group"
- Hidden spaces: "9th "
- Missing values silently dropped
- = (assignment) vs == (test)

 AI writes code fast – **you** are the one who checks it against the data.

2:45 – 3:00 · Scientific Storytelling

Report your DV mean in *one* sentence

Each team reports their filtered DV mean to the class in **one sentence**:

*"Among **[population]**, the average **[DV]** was **[X]**."*

*"Among **9th graders**, the average **test score** was **79.6**."*

As a class, discuss:

- Does this number **surprise** you?
- Does it **match** what the literature found this morning?

 This one sentence is the **seed of your Results section**.

 A good result is **one clear number** the audience can repeat back to you.

